마이크로프로세서

Chapter 7. Serial Communication

Human-Robot Interaction Laboratory

KYUNG HEE UNIVERSITY

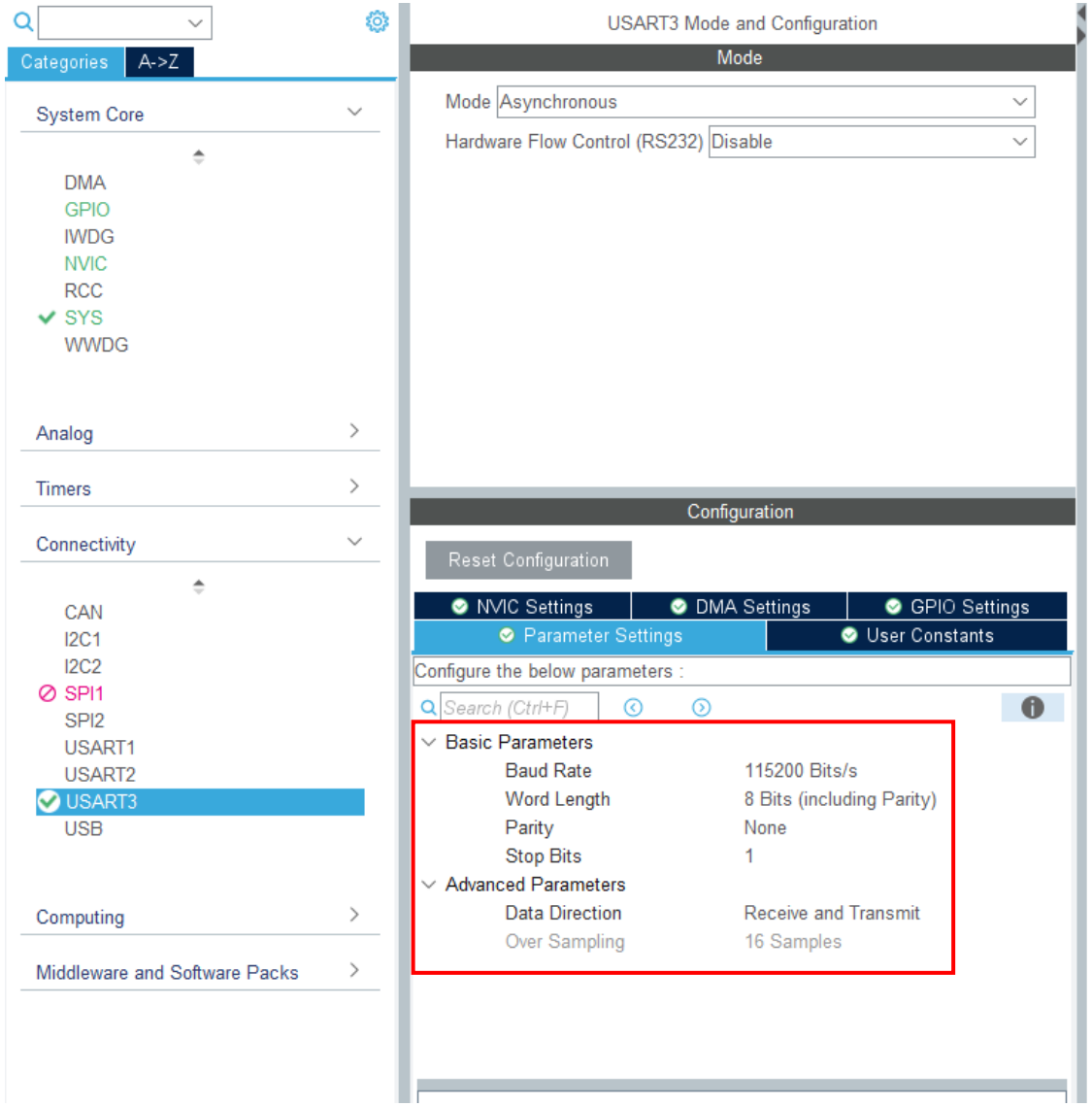
USART

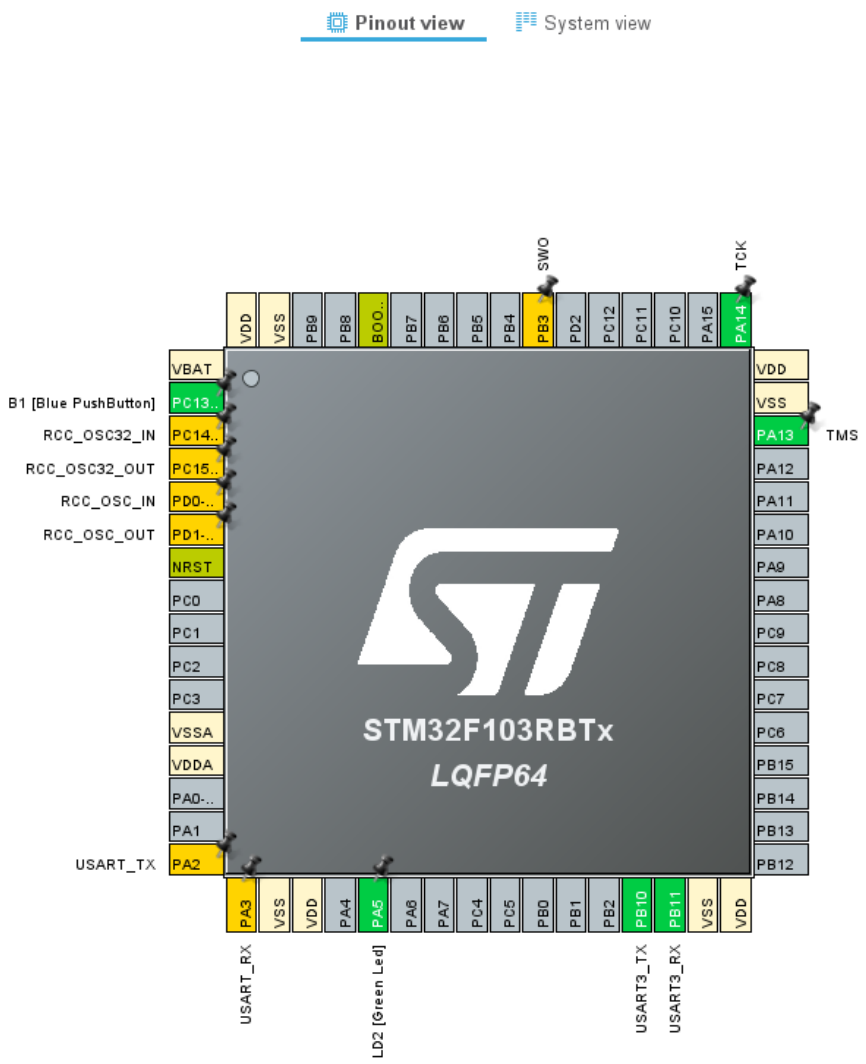
>> Universal Asynchronous Receiver/Transmitter

- Universal Synchronous/Asynchronous Receiver/Transmitter
- 기본적인 직렬 통신 방식
- 동기식 – 클럭을 통해 송/수신 타이밍을 조절, 빠르지만 하드웨어 복잡
- 비동기식 – 송/수신이 독립적으로 동작, 하드웨어가 단순, 데이터 손실 가능성이 있음
- 마이크로컨트롤러, 임베디드, IoT, 산업등 다양한 분야에서 활용

USART

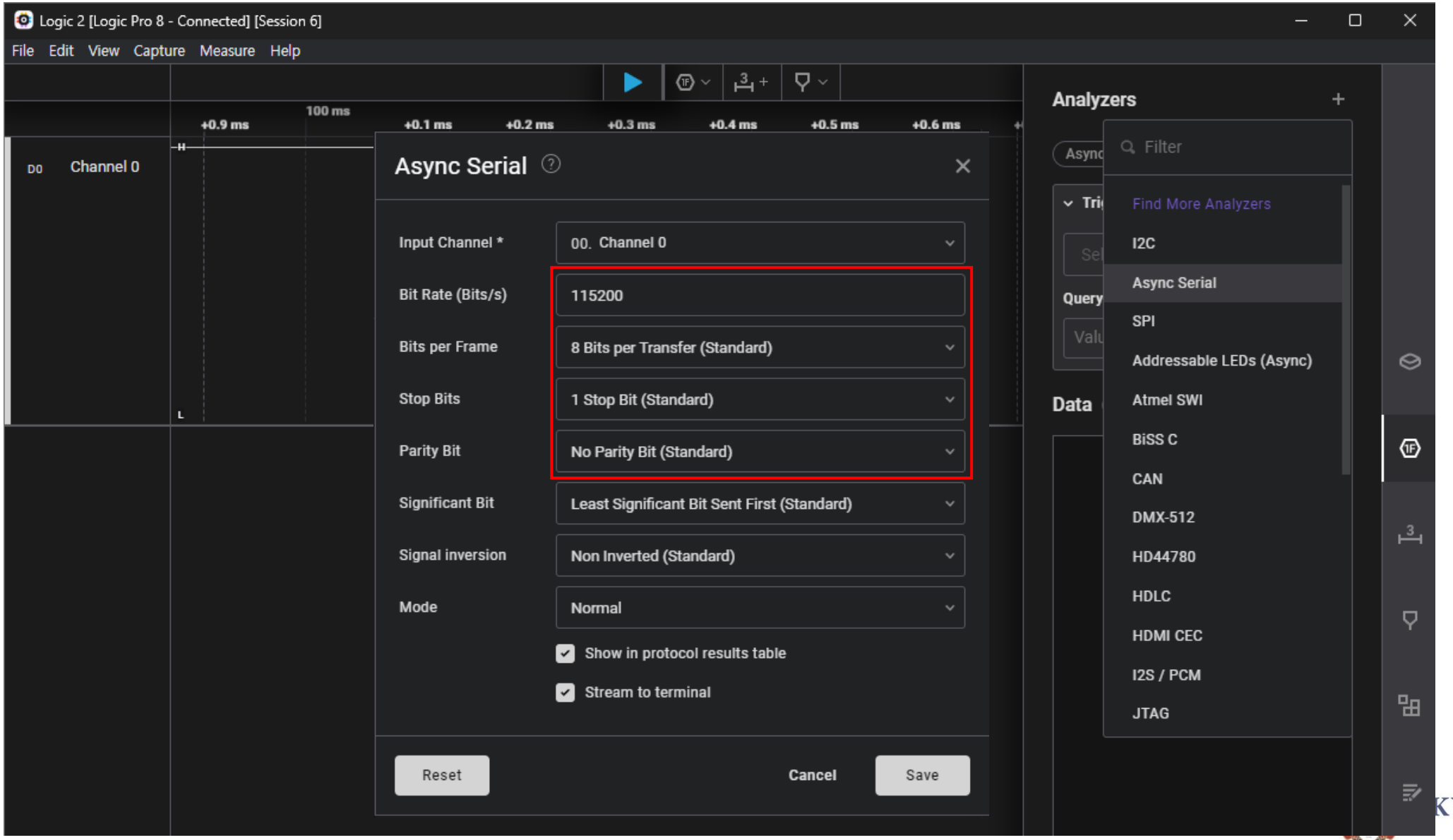
>> Universal Asynchronous Receiver/Transmitter





USART

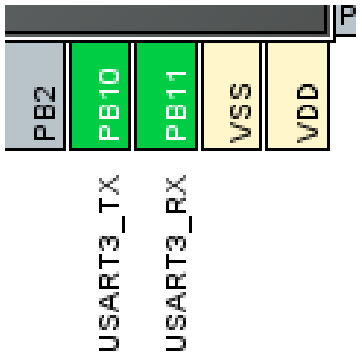
>> Universal Asynchronous Receiver/Transmitter

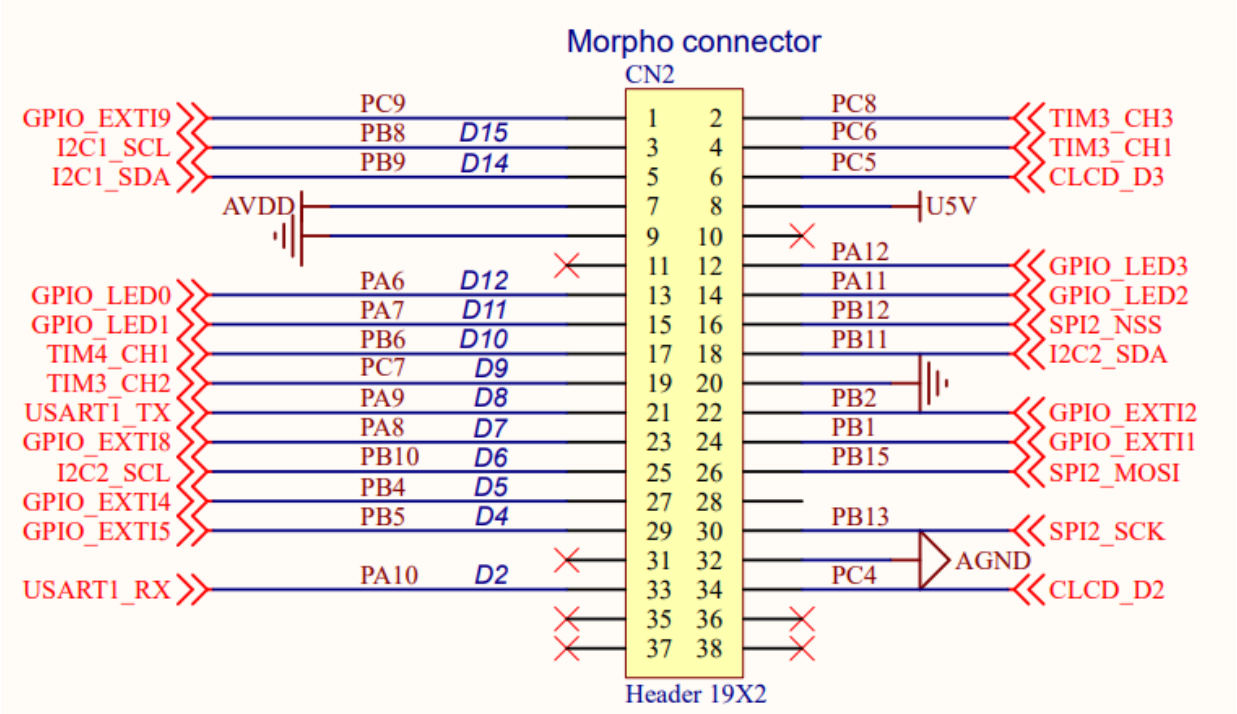


USART

>> Universal Asynchronous Receiver/Transmitter

- **USART3의 TX핀을 설정한 로직분석기 채널과 연결**



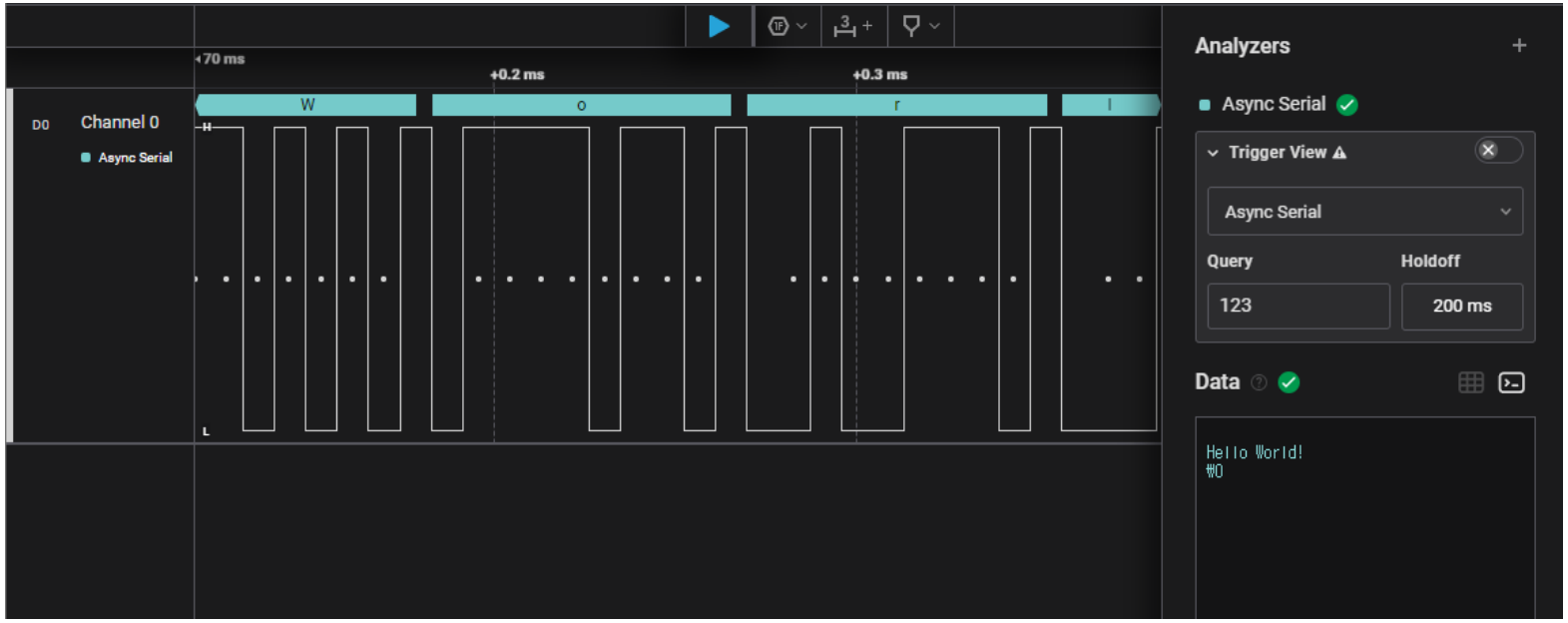


USART

>> Universal Asynchronous Receiver/Transmitter

- **USART를 통한 출력 확인**

```
uint8_t Trans[] = "Hello World!\n\r";
uint8_t buffsize = sizeof(Trans)/sizeof(uint8_t);

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
    HAL_UART_Transmit(&huart3, Trans, buffsize, 100);
    HAL_Delay(1000);
}
```

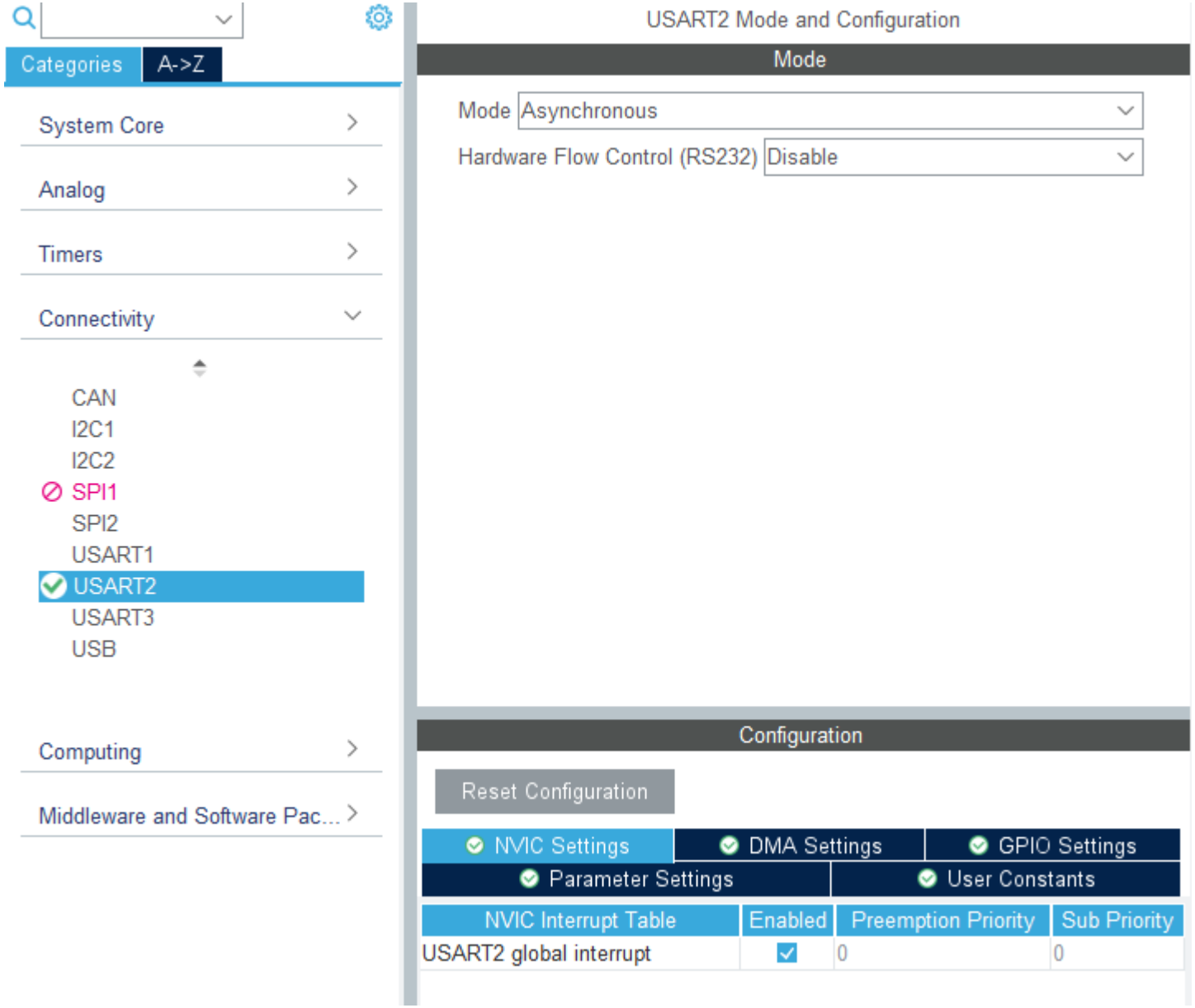


USART

>> Universal Asynchronous Receiver/Transmitter

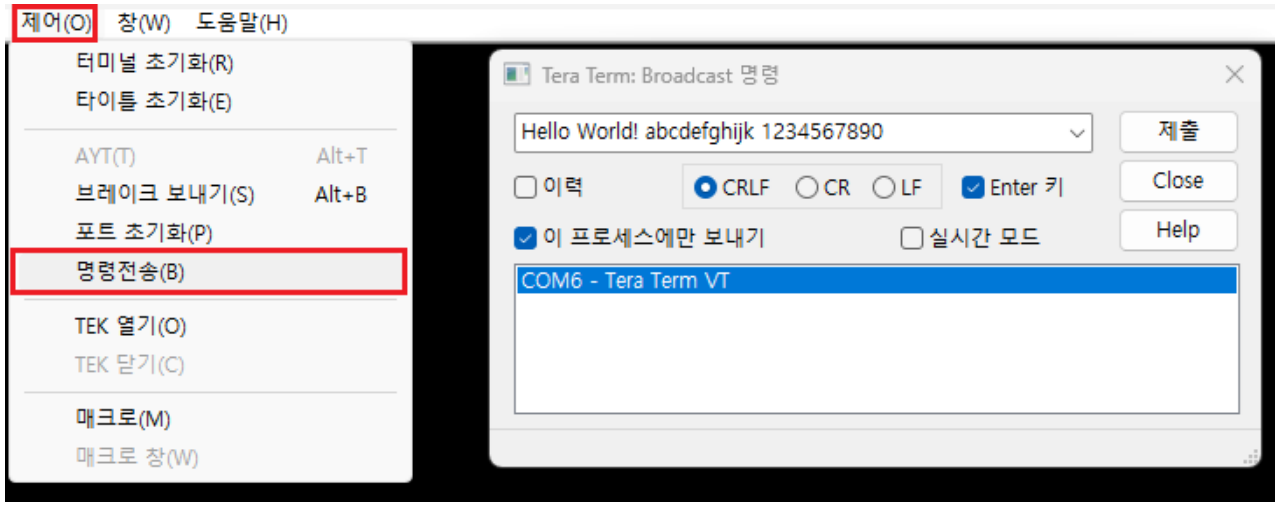
- 인터럽트 기반 UART Echo

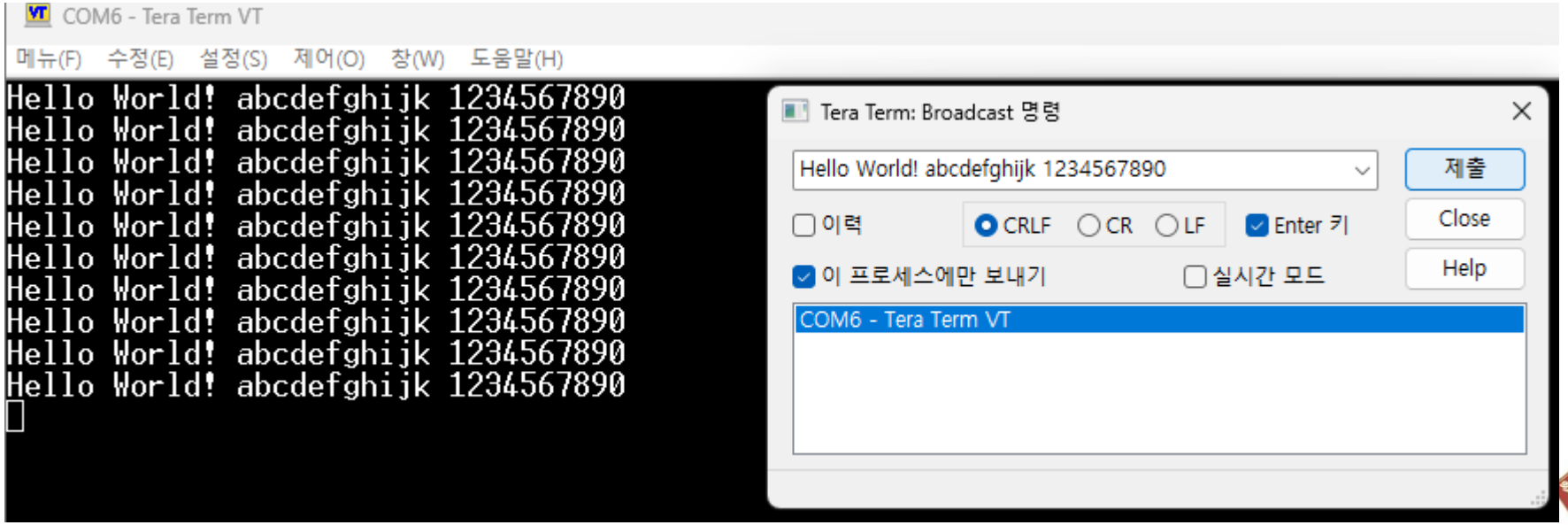
```c
/* USER CODE BEGIN PV */

#define BUFFSIZE 100
uint8_t RxBuffer[BUFFSIZE];
uint8_t RxIdx = 0;

/* USER CODE END PV */
  /* USER CODE BEGIN 2 */

  HAL_UART_Receive_IT(&huart2, &RxBuffer[RxIdx], 1);

  /* USER CODE END 2 */
/* USER CODE BEGIN 4 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if(huart->Instance == USART2) {
        if(RxBuffer[RxIdx] == '\n' || RxIdx > BUFFSIZE) {
            RxBuffer[RxIdx] = '\0';                          //End of data
            HAL_UART_Transmit(&huart2, RxBuffer, RxIdx, 100);
            HAL_UART_Transmit(&huart2, "\n", 1, 100);
            RxIdx = 0;
        }
        else
            RxIdx++;

        HAL_UART_Receive_IT(&huart2, &RxBuffer[RxIdx], 1);
    }
}
```



USART

>> Universal Asynchronous Receiver/Transmitter





I2C

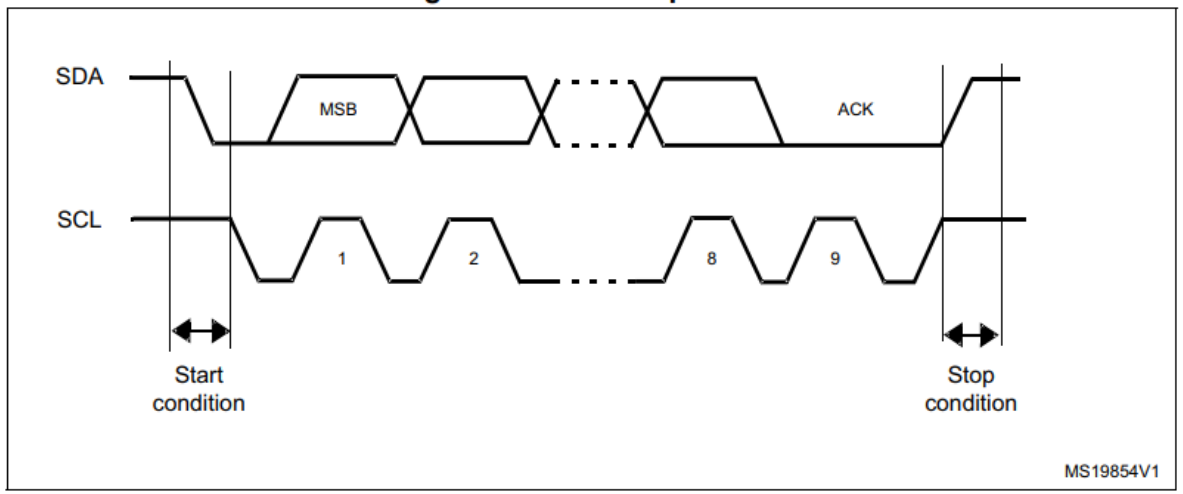
>> Inter-Integrated Circuit

- 하나의 데이터 라인을 통해 양방향 통신
- 여러개의 마스터와 슬레이브 연결 가능
- 마스터의 SCL(Serial Clock Line, 클럭) 제공을 통한 동기화 및 SDA(Serial Data Line, 데이터)를 통한 데이터 송수신, 2개의 신호선만을 사용하여 하드웨어가 단순함
- 마스터의 SCL을 기반으로 통신하여 동기식 데이터 전송
- SCL/SDA 두 라인에 슬레이브를 추가 연결 가능, 7/10비트 주소를 통해 통신

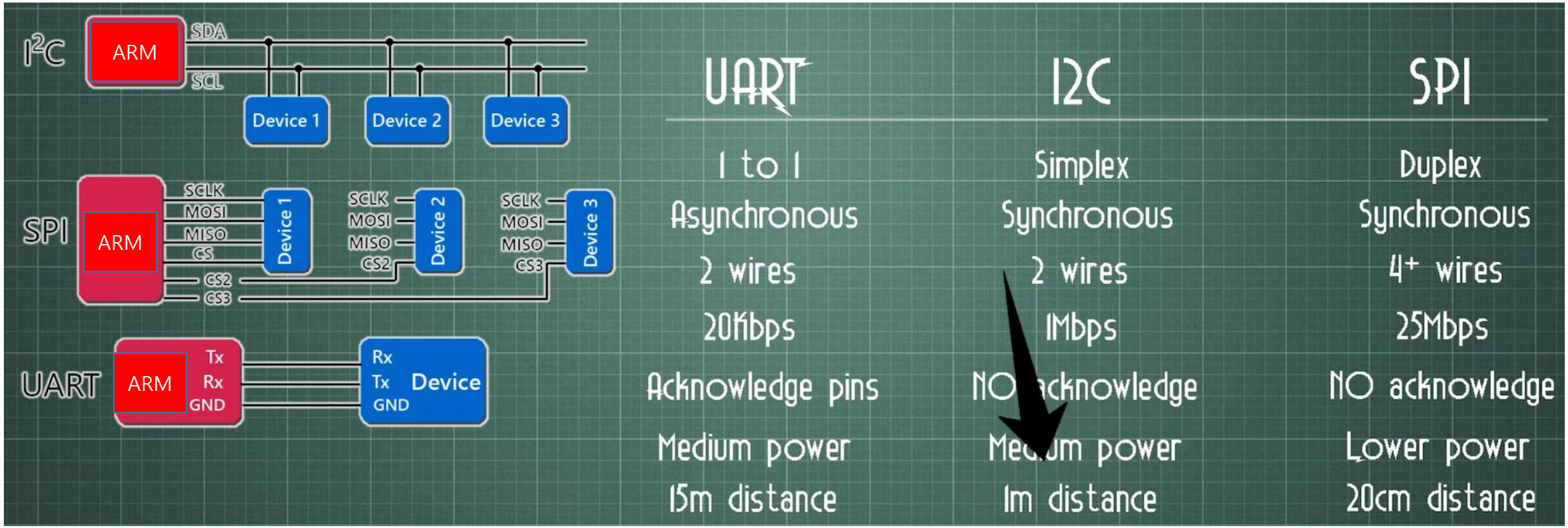
>> I2C Protocol

- I2C 통신은 SCL이 HIGH인 상태에서 SDA의 Falling Edge를 통해 통신을 시작
- 마스터에서 7비트 혹은 10비트의 슬레이브 주소를 전송
- 그 뒤에 읽기 또는 쓰기 비트를 추가하여 1바이트/8비트 데이터를 전송
- 슬레이브는 전송된 데이터의 주소가 자신에게 해당되면 ACK를 전송
- 마스터가 SCL이 HIGH인 상태에서 SDL의 Rising Edge를 통해 통신을 종료

Figure 269. I^{2}C bus protocol

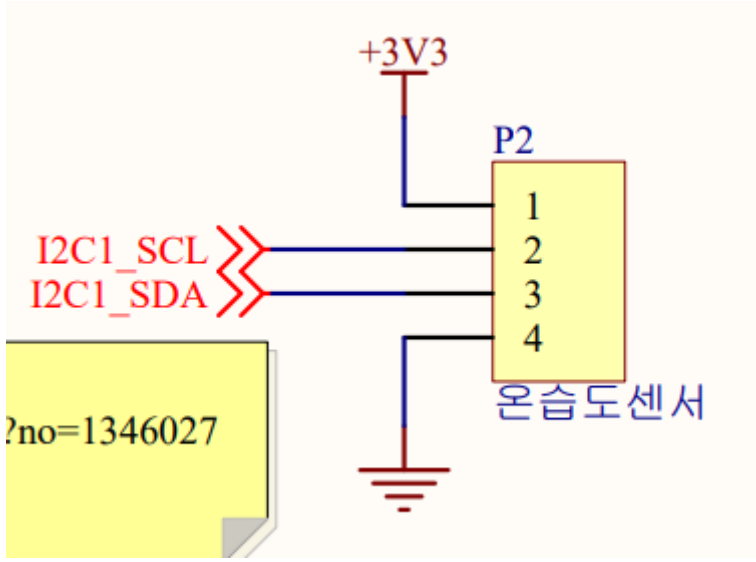


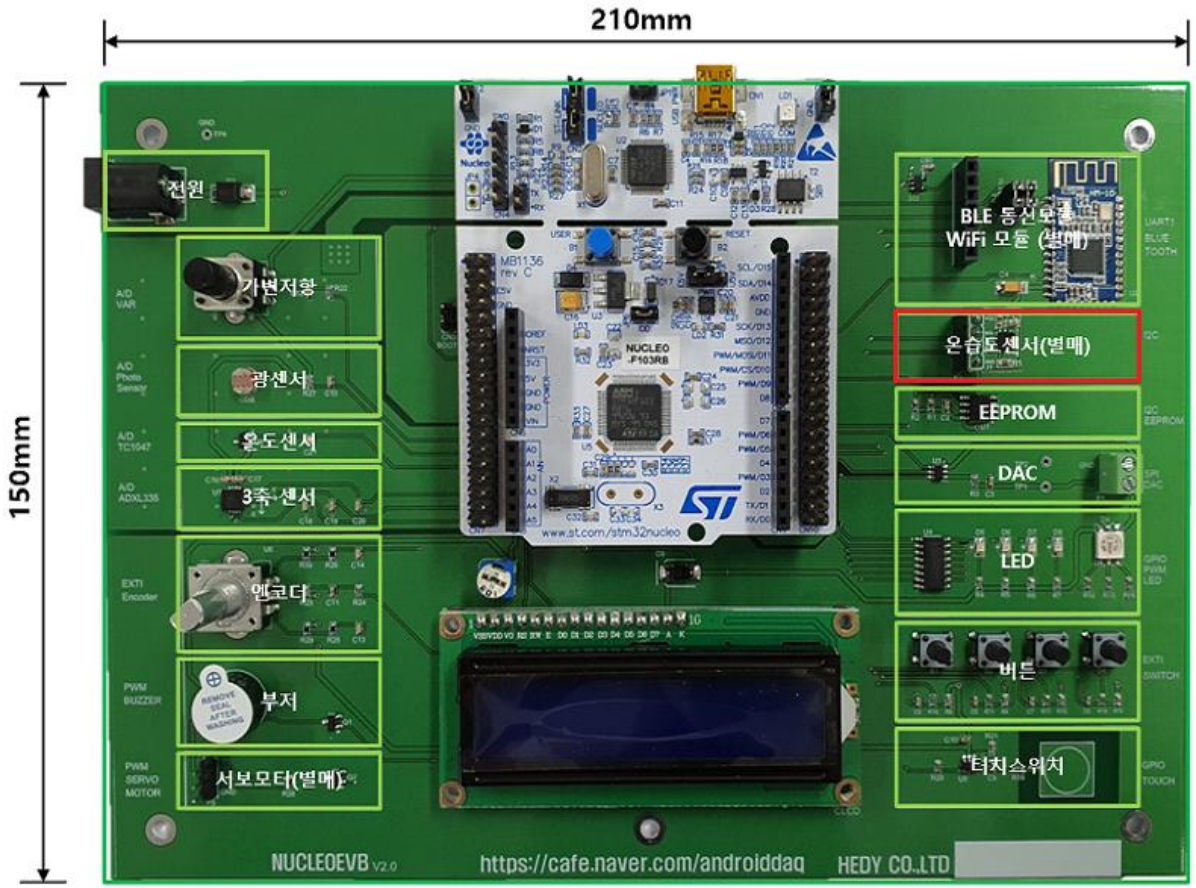
UART, I²C, SPI 비교



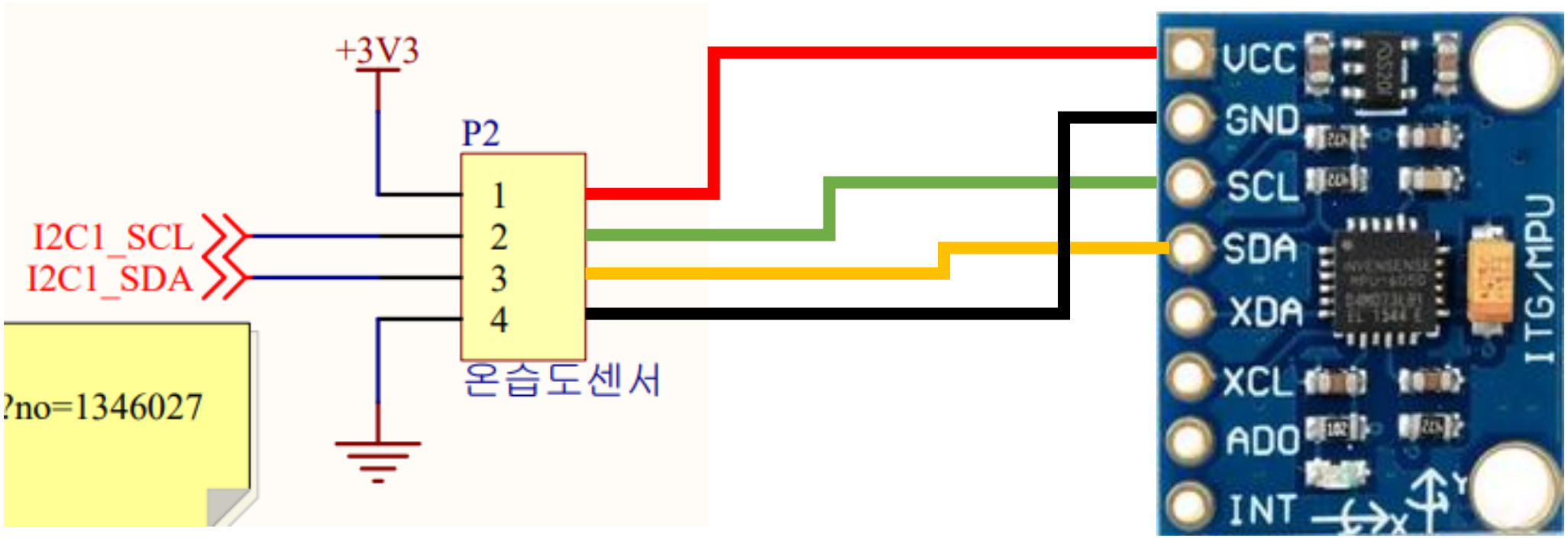
>> I2C

- MPU6050은 6축 가속도/자이로 센서, I2C 통신 프로토콜을 사용
- 점퍼선을 통해 확장보드 우측 I2C1 통신 포트에 점퍼선으로 센서 연결





>> I2C


+3V3
P2
1
2
3
4
I2C1_SCL
I2C1_SDA
온습도센서
?no=1346027
VCC
GND
SCL
SDA
XDA
XCL
ADO
INT
ITG/MPU

I2C

>> I2C

Pinout & Configuration | Clock Configuration | Project Manager

Software Packs | Pinout

Categories | A->Z

System Core
Analog
Timers
Connectivity
- CAN
- I2C1
- I2C2
- SPI1
- SPI2
- USART1
- USART2
- USART3
- USB

Computing
Middleware and Software Packs

I2C1 Mode and Configuration

Mode

I2C: I2C

Configuration

Reset Configuration

NVIC Settings | DMA Settings | GPIO Settings | Parameter Settings | User Constants

Configure the below parameters :

Search (Ctrl+F)

Master Features	
I2C Speed Mode	Standard Mode
I2C Clock Speed (Hz)	100000
Slave Features	
Clock No Stretch Mode	Disabled
Primary Address Length sel...	7-bit
Dual Address Acknowledged	Disabled
Primary slave address	0

Pinout view

I2C1_SDA
I2C1_SCL

VDD | VSS | PB9 | PB8 | BOOT0 | PB7 | PB6

VBAT
PC13..
PC14..
PC15..
PD0-..
PD1-..
NRST
PC0
PC1
PC2
PC3
VSSA

PushButton]
_OSC32_IN
OSC32_OUT
CC_OSC_IN
_OSC_OUT

STM32

KYUNG HEE UNIVERSITY

>> Inter-Integrated Circuit

```c
void MPU6050_Init(void) {
    uint8_t data[2];

    data[0] = 0x6B;
    data[1] = 0x00;
    HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR, data[0], 1, &data[1], 1, 100);

    data[0] = 0x1C;
    data[1] = 0x00;
    HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR, data[0], 1, &data[1], 1, 100);

    data[0] = 0x19;
    data[1] = 0x07;
    HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR, data[0], 1, &data[1], 1, 100);
}

void MPU6050_ReadData(int16_t* accel_data, int16_t* gyro_data) {
    uint8_t data[14];

    HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR, 0x3B, 1, data, 14, 100);

    accel_data[0] = (int16_t)((data[0] << 8) | data[1]);
    accel_data[1] = (int16_t)((data[2] << 8) | data[3]);
    accel_data[2] = (int16_t)((data[4] << 8) | data[5]);

    gyro_data[0] = (int16_t)((data[8] << 8) | data[9]);
    gyro_data[1] = (int16_t)((data[10] << 8) | data[11]);
    gyro_data[2] = (int16_t)((data[12] << 8) | data[13]);
}
```

```c
void MPU6050_Init(void) {
    uint8_t data[2]; // 8비트 정수 2개를 담을 배열 선언 (레지스터 주소 + 데이터)

    // 1. MPU6050 전원 관리 레지스터 설정 (PWR_MGMT_1)
    data[0] = 0x6B; // PWR_MGMT_1 레지스터 주소
    data[1] = 0x00; // 레지스터에 쓸 값 (0x00: 슬립 모드 해제, 내부 8MHz 오실레이터 사
    // I2C 통신을 통해 MPU6050 (MPU6050_ADDR)의 0x6B 레지스터에 0x00 값을 씀
    HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR, data[0], 1, &data[1], 1, 100);

    // 2. MPU6050 가속도계 설정 (ACCEL_CONFIG)
    data[0] = 0x1C; // ACCEL_CONFIG 레지스터 주소
    data[1] = 0x00; // 레지스터에 쓸 값 (0x00: 가속도계 풀 스케일 범위 ±2g 로 설정)
    // I2C 통신을 통해 MPU6050의 0x1C 레지스터에 0x00 값을 씀
    HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR, data[0], 1, &data[1], 1, 100);

    // 3. MPU6050 샘플링 속도 설정 (SMPLRT_DIV)
    data[0] = 0x19; // SMPLRT_DIV 레지스터 주소
    data[1] = 0x07; // 레지스터에 쓸 값 (0x07: 샘플링 속도를 1kHz로 설정)
                    // Sample Rate = Gyroscope Output Rate / (1 + SMPLRT_DIV)
                    // 기본 자이로 출력 속도가 8kHz일 경우, 8kHz / (1 + 7) = 1kHz
    // I2C 통신을 통해 MPU6050의 0x19 레지스터에 0x07 값을 씀
    HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR, data[0], 1, &data[1], 1, 100);
}
```

I2C

>> Inter-Integrated Circuit

```c
void MPU6050_Init(void) {
    uint8_t data[2];

    data[0] = 0x6B;
    data[1] = 0x00;
    HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR, data[0], 1, &data[1], 1, 100);

    data[0] = 0x1C;
    data[1] = 0x00;
    HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR, data[0], 1, &data[1], 1, 100);

    data[0] = 0x19;
    data[1] = 0x07;
    HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR, data[0], 1, &data[1], 1, 100);
}

void MPU6050_ReadData(int16_t* accel_data, int16_t* gyro_data) {
    uint8_t data[14];

    HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR, 0x3B, 1, data, 14, 100);

    accel_data[0] = (int16_t)((data[0] << 8) | data[1]);
    accel_data[1] = (int16_t)((data[2] << 8) | data[3]);
    accel_data[2] = (int16_t)((data[4] << 8) | data[5]);

    gyro_data[0] = (int16_t)((data[8] << 8) | data[9]);
    gyro_data[1] = (int16_t)((data[10] << 8) | data[11]);
    gyro_data[2] = (int16_t)((data[12] << 8) | data[13]);
}
```

```c
// MPU6050으로부터 가속도 및 자이로 데이터를 읽는 함수
// accel_data: 가속도 X, Y, Z 값을 저장할 배열 포인터
// gyro_data: 자이로 X, Y, Z 값을 저장할 배열 포인터
void MPU6050_ReadData(int16_t* accel_data, int16_t* gyro_data) {
    uint8_t data[14]; // MPU6050으로부터 읽어올 14바이트 데이터를 저장할 배열

    // 1. MPU6050 센서 데이터 읽기
    // HAL_I2C_Mem_Read 함수를 사용하여 I2C 통신으로 MPU6050의 특정 레지스터부터 데이터
    // &hi2c1: I2C 핸들러
    // MPU6050_ADDR: MPU6050의 I2C 주소 (0xD0)
    // 0x3B: 읽기 시작할 MPU6050 내부 레지스터 주소 (ACCEL_XOUT_H)
    // 1: 레지스터 주소 크기 (1바이트)
    // data: 읽어온 데이터를 저장할 버퍼
    // 14: 읽어올 데이터 크기 (14바이트). MPU6050은 0x3B부터 순차적으로 가속도(6바이트),
    //     온도(2바이트), 자이로(6바이트) 데이터를 저장하고 있음. 따라서 14바이트를 읽으
    //     이 모든 데이터를 한 번에 가져올 수 있음.
    // 100: 타임아웃 (100ms)
    HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR, 0x3B, 1, data, 14, 100);

    // 2. 가속도 데이터 처리 (상위 바이트 + 하위 바이트 결합)
    // MPU6050은 각 축의 데이터를 16비트 (2바이트)로 제공하며, 상위(High) 바이트와
    // 하위(Low) 바이트로 나누어져 있음. 이 두 바이트를 합쳐 하나의 16비트 값으로 만듦.
    // (data[0] << 8): 상위 바이트를 왼쪽으로 8비트 시프트
    // | data[1]: 하위 바이트와 비트 OR 연산 수행
    // (int16_t): 결과를 부호 있는 16비트 정수형으로 변환
    accel_data[0] = (int16_t)((data[0] << 8) | data[1]); // X축 가속도 (data[0]=ACC
    accel_data[1] = (int16_t)((data[2] << 8) | data[3]); // Y축 가속도 (data[2]=ACC
    accel_data[2] = (int16_t)((data[4] << 8) | data[5]); // Z축 가속도 (data[4]=ACC

    // 3. 자이로 데이터 처리 (상위 바이트 + 하위 바이트 결합)
    // data[6]과 data[7]은 온도 데이터이므로 건너뛰고, data[8]부터 자이로 데이터를 처리
    gyro_data[0] = (int16_t)((data[8] << 8) | data[9]);   // X축 자이로 (data[8]=GY
    gyro_data[1] = (int16_t)((data[10] << 8) | data[11]);// Y축 자이로 (data[10]=GY
    gyro_data[2] = (int16_t)((data[12] << 8) | data[13]);// Z축 자이로 (data[12]=GY
}
```

>> Inter-Integrated Circuit

```c
/* USER CODE BEGIN 2 */

MPU6050_Init();

int16_t accel[3], gyro[3];
float f_accel[3], f_gyro[3];

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */

    MPU6050_ReadData(accel, gyro);

    for(int i = 0; i < 3; i++) {
        f_accel[i] = (float)accel[i] / 16384.0f;
        f_gyro[i]  = (float)gyro[i]  / 131.0f;
    }

    printf("Accel: %f, %f, %f\n", f_accel[0], f_accel[1], f_accel[2]);
    printf("Gyro : %f, %f, %f\n\n", f_gyro[0], f_gyro[1], f_gyro[2]);

    HAL_Delay(500);
}
/* USER CODE END 3 */
```

```c
f_accel[i] = (float)accel[i] / 16384.0f;  // LSB/g → g 단위로 변환
f_gyro[i] = (float)gyro[i] / 131.0f;      // LSB/(°/s) → °/s 단위로 변환
```

변환 계수의 의미

- **16384:** MPU6050의 가속도계 기본 설정(±2g 범위)에서 1g당 LSB 값
- **131:** 자이로스코프 기본 설정(±250°/s 범위)에서 1°/s당 LSB 값

>> Inter-Integrated Circuit

- **MPU6050은 7비트 주소를 사용**
  - #define MPU6050_ADDR 0xD0 //0x68 << 1
  - bit0은 사용되지 않아 주소를 shift 하여 사용
- **MPU6050 init에서 센서의 Sleep 모드 해제 및 센서 측정 계수 설정**
- **센서 계수 설정은 MPU6050 Register map 참조**

Addr (Hex)	Addr (Dec.)	Register Name	Serial I/F	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
67	103	I2C_MST_DELAY_CTRL	R/W	DELAY_ES_SHADOW	-	-	I2C_SLV4_DLY_EN	I2C_SLV3_DLY_EN	I2C_SLV2_DLY_EN	I2C_SLV1_DLY_EN	I2C_SLV0_DLY_EN
68	104	SIGNAL_PATH_RESET	R/W	-	-	-	-	-	GYRO_RESET	ACCEL_RESET	TEMP_RESET
6A	106	USER_CTRL	R/W	-	FIFO_EN	I2C_MST_EN	I2C_IF_DIS	-	FIFO_RESET	I2C_MST_RESET	SIG_COND_RESET
6B	107	PWR_MGMT_1	R/W	DEVICE_RESET	SLEEP	CYCLE	-	TEMP_DIS	CLKSEL[2:0]		
19	25	SMPLRT_DIV	R/W	SMPLRT_DIV[7:0]							
1A	26	CONFIG	R/W	-	-	EXT_SYNC_SET[2:0]			DLPF_CFG[2:0]		
1B	27	GYRO_CONFIG	R/W	-	-	-	FS_SEL [1:0]		-	-	-
1C	28	ACCEL_CONFIG	R/W	XA_ST	YA_ST	ZA_ST	AFS_SEL[1:0]				

26.6.3 I²C Own address register 1 (I2C_OAR1)

Address offset: 0x08
Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD MODE	Reserved					ADD[9:8]		ADD[7:1]							ADD0
rw						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 15 **ADDMODE** Addressing mode (slave mode)
0: 7-bit slave address (10-bit address not acknowledged)
1: 10-bit slave address (7-bit address not acknowledged)

Bit 14 Should always be kept at 1 by software.

Bits 13:10 Reserved, must be kept at reset value

Bits 9:8 **ADD[9:8]**: Interface address
7-bit addressing mode: don't care
10-bit addressing mode: bits9:8 of address

Bits 7:1 **ADD[7:1]**: Interface address
bits 7:1 of address

Bit 0 **ADD0**: Interface address
7-bit addressing mode: don't care
10-bit addressing mode: bit 0 of address

>> Inter-Integrated Circuit

- MPU6050 init에서 HAL_I2C_Mem_Write를 통해 해당 센서 레지스터 주소에 1바이트씩 데이터 write

- MPU6050 ReadData에서 HAL _I2C_Mem_Read를 통해 센서 레지스터 0x3B부터 14바이트 읽어 들임

- 데이터는 1바이트씩 읽어 들이며, 각 축의 가속도, 자이로값의 상위/하위 비트로 구성되어 총 2바이트의 센서값을 구성함(0x41, 0x42는 센서 온도 센서 2바이트 데이터)

3B	59	ACCEL_XOUT_H	R	ACCEL_XOUT[15:8]
3C	60	ACCEL_XOUT_L	R	ACCEL_XOUT[7:0]
3D	61	ACCEL_YOUT_H	R	ACCEL_YOUT[15:8]
3E	62	ACCEL_YOUT_L	R	ACCEL_YOUT[7:0]
3F	63	ACCEL_ZOUT_H	R	ACCEL_ZOUT[15:8]
40	64	ACCEL_ZOUT_L	R	ACCEL_ZOUT[7:0]
41	65	TEMP_OUT_H	R	TEMP_OUT[15:8]
42	66	TEMP_OUT_L	R	TEMP_OUT[7:0]
43	67	GYRO_XOUT_H	R	GYRO_XOUT[15:8]
44	68	GYRO_XOUT_L	R	GYRO_XOUT[7:0]
45	69	GYRO_YOUT_H	R	GYRO_YOUT[15:8]
46	70	GYRO_YOUT_L	R	GYRO_YOUT[7:0]
47	71	GYRO_ZOUT_H	R	GYRO_ZOUT[15:8]
48	72	GYRO_ZOUT_L	R	GYRO_ZOUT[7:0]

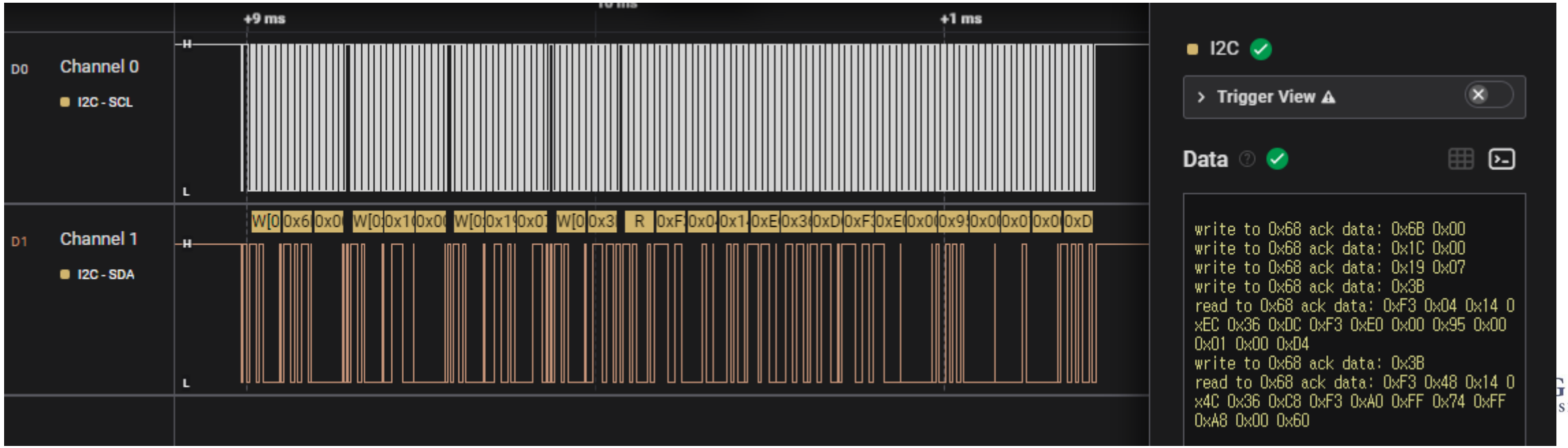
I2C

>> Inter-Integrated Circuit

- 로직 분석기를 통해 센서의 통신을 확인

- 센서의 주소인 0x68에서 0x6B/0x1C/0x19 주소에 센서 설정 비트 전송

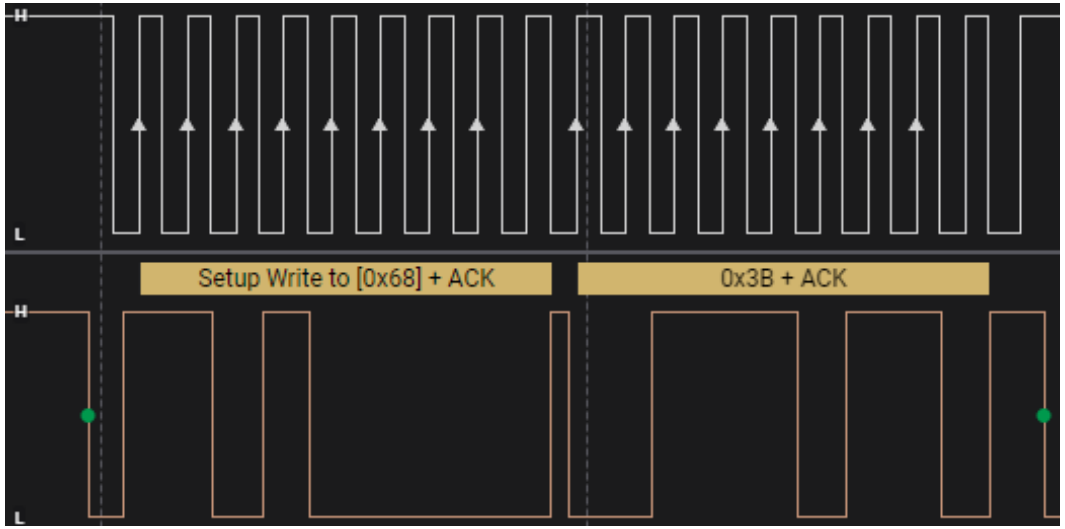
- Sleep 모드 해제, 센서 측정 범위, 측정 계수 설정

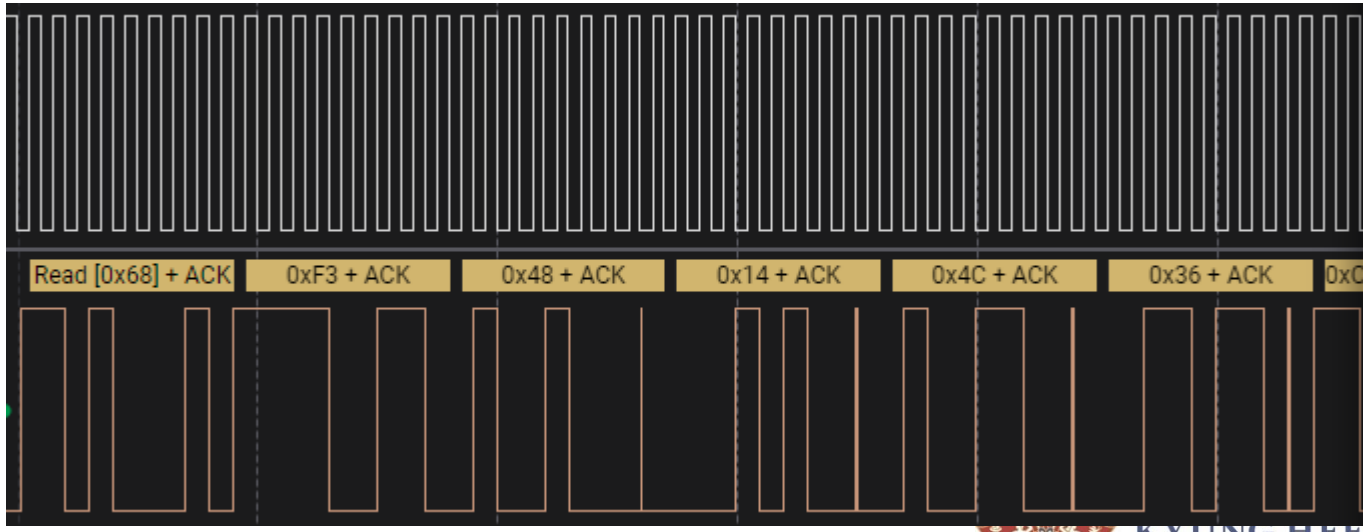


I2C

>> Inter-Integrated Circuit

- 로직 분석기를 통해 센서의 통신을 확인
- 센서의 주소인 0x68에서 0x3B 주소를 전송
- 이후 14바이트의 데이터를 수신 받음





I2C

>> Inter-Integrated Circuit

- 가속도는 16비트 정수를 16384로 나누어 ±2g(중력 가속도) 범위의 값을 측정
- 센서를 올바르게 둘 경우 Z 축에 약 1.0g가 측정, 다양한 각도로 센서를 두어 중력가속도 확인
- 각속도 역시 16비트 정수로 읽어 들이며, 131로 나누어 ±250°/s 범위의 값을 측정
- 축에 따라 회전 속도를 확인

AFS_SEL	Full Scale Range	LSB Sensitivity
0	±2*g*	16384 LSB/*g*
1	±4*g*	8192 LSB/*g*
2	±8*g*	4096 LSB/*g*
3	±16*g*	2048 LSB/*g*

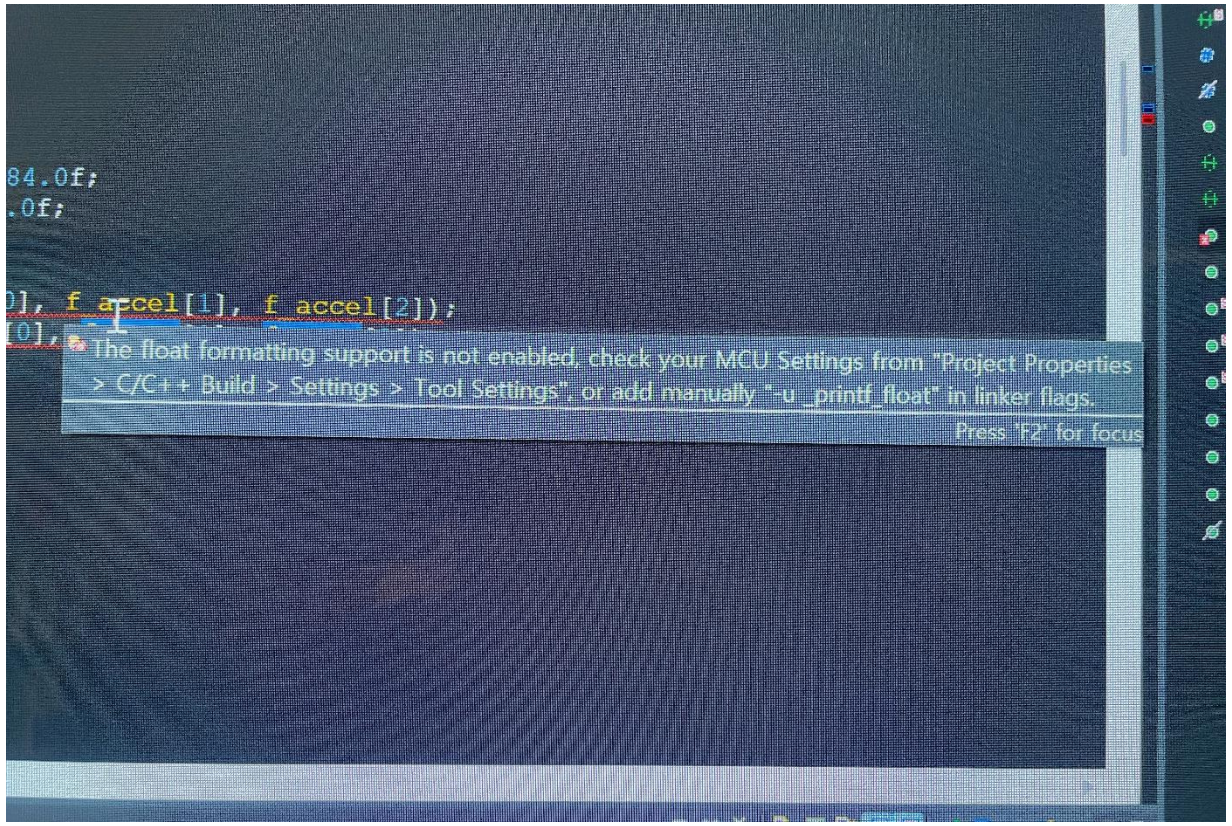
FS_SEL	Full Scale Range	LSB Sensitivity
0	± 250 °/s	131 LSB/°/s
1	± 500 °/s	65.5 LSB/°/s
2	± 1000 °/s	32.8 LSB/°/s
3	± 2000 °/s	16.4 LSB/°/s

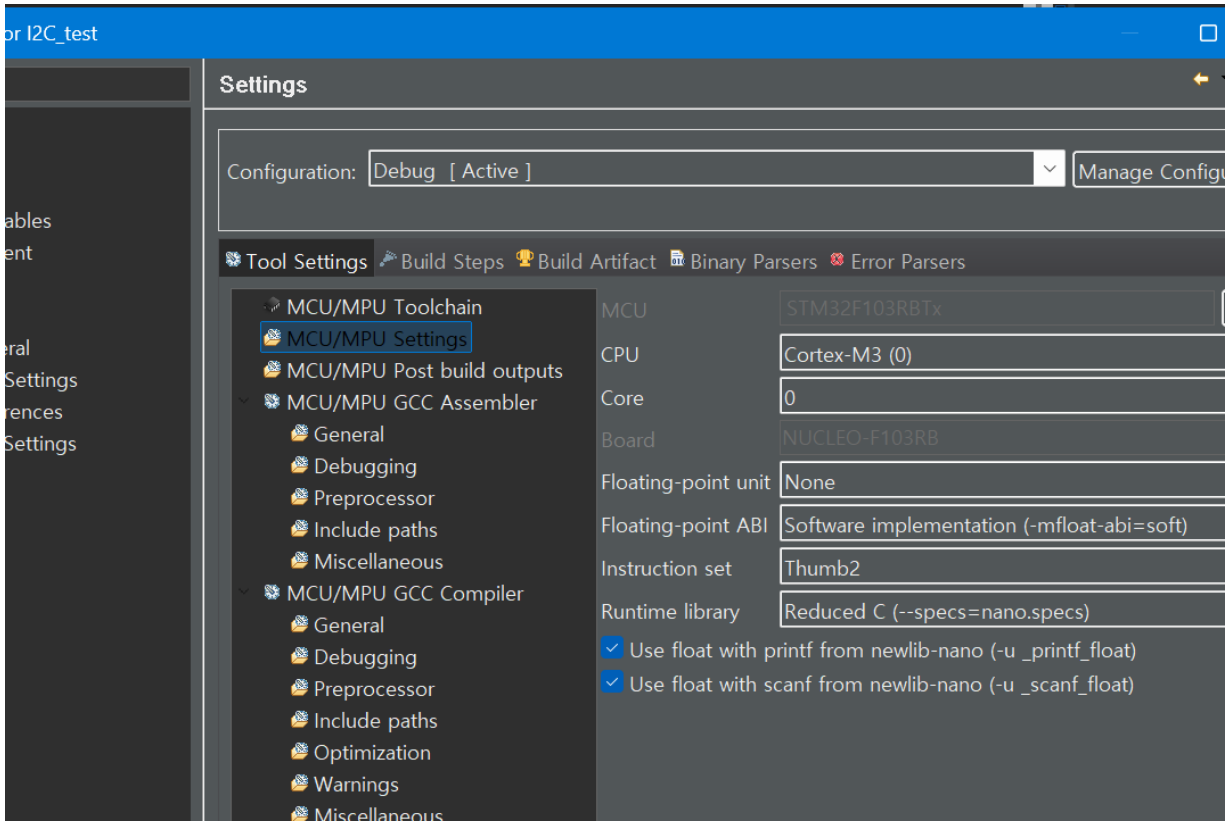
COM6 - Tera Term VT

메뉴(F) 수정(E) 설정(S) 제어(O) 창(W) 도움말(H)

```
Accel: 0.029297, -0.107178, 0.946289
Gyro : -3.152672, 3.603053, 0.916031

Accel: -0.058105, -0.120605, 0.952637
Gyro : -0.618321, -17.251909, -0.977099

Accel: 0.025879, -0.103271, 0.950684
Gyro : -2.083969, -11.885496, 7.893130

Accel: 0.031006, -0.018311, 0.952637
Gyro : 14.702291, -11.725191, 7.809160

Accel: 0.101562, -0.166992, 0.959229
Gyro : 16.335878, 14.748092, 0.412214

Accel: 0.041260, -0.056885, 0.981445
Gyro : -0.725191, 4.122138, -17.106871

Accel: 0.090088, -0.116699, 0.967773
Gyro : 4.000000, -9.389313, 17.412214

Accel: 0.074463, -0.143799, 0.963867
Gyro : -0.732824, -0.633588, 0.282443

Accel: 0.093506, -0.134277, 0.960205
Gyro : -1.076336, 1.580153, 3.572519

Accel: 0.089600, -0.129150, 0.953369
Gyro : -0.770992, -0.847328, 2.045802
```

시리얼모니터(teta term)에서 숫자 안보일 때





Thank you

Department of Electronic Engineering,
Kyung Hee University, 1732, Deogyeong-Daero,
Giheung-gu, Yongin-si, Gyeonggi-Do, 17104,
Republic of Korea
Web: https://khu-hri.weebly.com/

Human-Robot Interaction Laboratory

KYUNG HEE UNIVERSITY